Devaj Rathore
2023190

CSE 112 Computer Organization: Quiz 2 (SET A)

Total Time: 45 Minutes

Maximum: 45 Marks

Instructions:

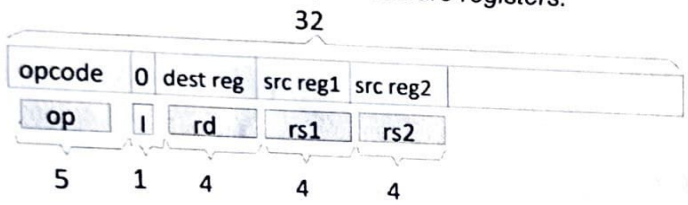
1. **Write the SET name on the top of the first page of answer script for it to get evaluated.**
2. **Mobile phones, bags, and wristwatches of any kind are NOT Permitted in the examination hall.**
3. **Write all the assumptions clearly, if any. *Only reasonable assumptions will be considered if any ambiguity is found in the question*.**
4. **Institute's plagiarism policy will apply if any unfair means are observed.**
5. **Use of non-programmable calculators is permitted.**

Q1. 32-bit split for the following instructions corresponding to an ISA having 16 registers and 21 instructions is below. Match the instruction with the correct encoding corresponding to that instruction: **[2*4 = 8 Marks]**

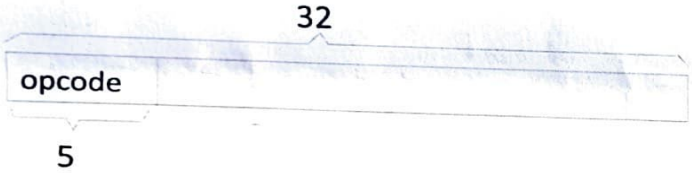
1. mov rd, rs (move contents of rs to rd)
2. nop (no operation)
3. add rd, rs1, rs2 (add the values in registers rs1+rs2 and store in rd)
4. ld rd, #imm[rs] (load rd with contents at memory location obtained by adding #imm and contents of rs)

Note: *Wherever explicitly visible, the 6th bit equal to 1 in the instruction code means an immediate value is present as one of the operands. 0 means all operands are registers.*

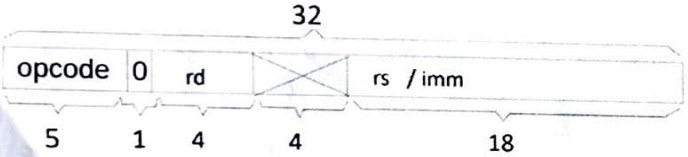
A.



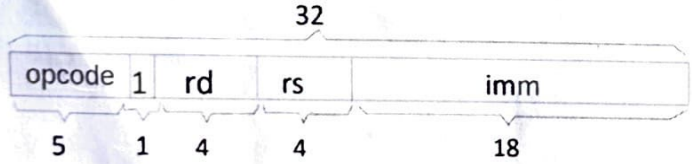
B.



C.



D.



Q2. Consider a stack that starts from memory location **0x3FF (hexadecimal)**. Consider the two instructions below for stack operations. **[1*10 = 10 Marks]**

Syntax	Semantics
PUSH reg1	Pushes the value of register reg1 into the stack.
POP reg1	Pops the value from the top of the stack into register reg1

Whenever a value is put into the stack via **PUSH** instruction, the value of the Stack Pointer (SP) **decrements** by one (binary). Whenever a value is moved out of the stack via **POP** instruction, the value of SP **increments** by one (binary). For the given program, determine the value of SP for each instruction.

Instructions:

1. POP r2
2. POP r2
3. PUSH r1
4. POP r1
5. POP r5
6. POP r4
7. PUSH r2
8. POP r3
9. PUSH r5
10. POP r7

Q3.Study the ISA given below and answer the questions that follow. **[17 Marks]**

	S. No.	Instruction (Mnemonic, Operands)	Operation Performed
0	1.	ADD R1, R2, R3	Add contents of registers R2 and R3 and store the result in R1.
1	2.	SUBI R1, #Imm	Subtract the content of the specified register(R1) by immediate value (imm).
10	3.	BNZ R1, Address	Branch to address if the content of specified register(R1) is not equal to zero.
11	4.	MUL R1, R2	Multiply the contents of registers R1, R2 and store the result in the specified register, R1.
00	5.	MOV R1, R2	Copy the content of the register R2 into the register R1.
	6.	LD R1, Address	Load the content specified at the mentioned Address into the register (R1).
	7.	ST R1, Address	Store the content of register R1 to the specified address.
	8.	MVI R1, #Imm	Copy the Immediate value into the specified register (R1).
	9.	BL Address	Branch to the mentioned address and store the return address (address of next instruction) in the Link register.

(a) Study the pseudo-code and corresponding assembly code given below and fill in the blanks (marked as "**BLANK**") based on the given ISA, pseudo-code, and associated assumptions. You can fill in assembly instructions or immediate value or register in the blanks, whichever is correct. **[6*2 = 12 Marks]**

Assume the following for the assembly code:

1. All registers are initialized to value 0 by default. There are 16 registers (r0 to r15) in total.
2. The final result will be a quotient retaining only the integral value. E.g. for dividend 5 and divisor 2, the assembly code should give 2 as a result (quotient) rather than 2.5.

***Note:** "//" indicates the beginning of an in-line comment.*

P.T.O.

Pseudo-code:

```
main:
    temp_number = number_which_has_to_be_divided_by_2
    quotient_value = 0
    final_answer = sub_again (temp_number)
    store final_answer at roll_number
sub_again:
    temp_number = temp_number - 2
    quotient_value = quotient_value + 1
    if (temp_number == 0) then // for even number, repeated subtraction by 2 results in remainder = 0.
            return quotient_value and go back to the caller
    temp_number_2 = temp_number - 1 // for odd number, repeated subtraction by 2 results in remainder = 1.
    if(temp_number_2 ==0) then // checking if remainder - 1 == 0 for odd number.
            return quotient_value and go back to the caller
    call sub_again
end
```

Assembly-code: ***(line numbers do not imply addresses; they are just for visual clarity.)***

```
main:
1.    mvi r3, #1
2.    mvi BLANK, #10 // Here, the dividend is taken to be 10.
3.    BLANK sub_again // calling sub_again.
4.    st r14, #roll_no // storing the quotient in #roll_no.

sub_again:
5.    subi r15, #2 // repeated subtraction of dividend with divisor.
6.    add  r14, r14 BLANK // iterating the quotient.
7.    mov  r13, r15
8.    BLANK  r15, sub_again // For even no., dividend will become 0 in last iteration and this loop won't get reiterated.
9.    subi r13, BLANK
10.   bnz  r13, BLANK // For odd no., r13 will become 0 in last iteration and this loop won't get reiterated.
11.   mov r10, r7 // Updating PC (program-counter) to return back to the caller.
```

(b) Report all the possible caller function(s) and callee function(s) in your code. **[1*2 = 2 Marks]**

(c) After studying the assembly code, identify the registers corresponding to the program counter (PC) and link register. **[1.5*2 = 3 Marks]**

Q4. Using the ISA in Q3, write an assembly code to implement the given pseudo-code: **[5 Marks]**

Pseudo-code: ***(line numbers do not imply addresses; they are just for visual clarity.)***

```
1. main:
2.      number1 = last_digit_of_your_roll_number + 1
3.      number2 = last_second_digit_of_your_roll_number
4.      number3 = number1 + number2
5. label1:
6.      number1=number1 + number2
7.      number3=number3 - 1
8.      if(number3 !=0) then
9.              go_to label1
10. exit
```

Q5. A 32-bit ISA has 16 registers (R0, R1..) and 21 instructions with encoding same as in Q1. Of these 21 instructions, the first 9 are provided in a table in Q3. Decode the binary instruction codes provided below and identify the operation being performed (opcode), the source registers, the destination registers, and the immediate value. **[5 Marks]**

Note: *The 6th bit equal to 1 in the instruction code means an immediate value is present as one of the operands. 0 means all operands are registers.*

1. 0010_0000_0110_0000_0000_0000_0000_0000
2. 0000_0010_0110_0000_0000_0000_0000_0000

Hint: *For e.g., for the 3rd instruction in the table in Q3, the 5-bit opcode will be = 00010 (BNZ).*